Mixing Quantum and AI: My Personal Research Journey

QMLHEP III TASK

Author: Rohit Chiluka - Undergrad Student (AIML Major)

Focus: Quantum Machine Learning (QML) & Using Genetic Algorithms to fix training issues

1. Why am I even looking at Quantum?

As an AIML student, I've spent countless hours training models with standard backpropagation. It works, but it feels like we're just making models bigger and bigger to get better results. I got curious about Quantum Computing because it's a totally different way of thinking about math.

Instead of just storing data in 0s and 1s, we use qubits. This lets us map our data into a "Hilbert Space"—which basically just means a crazy high-dimensional space where we can find patterns that a normal neural network might miss. It's not about replacing my GPU; it's about finding a better way to "see" complex data.

2. What I'm Learning Right Now

My Favorite Algorithm: QAOA

I've been looking into the **Quantum Approximate Optimization Algorithm (QAOA)**. It's one of the few quantum algorithms that actually works on the "noisy" quantum computers we have today.

- **How I see it:** It's like a hybrid loop. The quantum part sets up a "shape," and a classical optimizer (like the ones we use in ML) tweaks it until it finds the best answer.
- **The Goal:** It's perfect for "search" problems, like finding the fastest route for a delivery truck or figuring out how a protein folds.

The Software: PennyLane

I'm using **PennyLane** for my projects. If you know PyTorch or NumPy, it's actually pretty easy to pick up. It treats a quantum circuit like just another layer in a neural network, which is awesome because I don't have to be a physics expert to start building things.

3. My Big Idea: Using Evolution to Build Circuits

The biggest problem I've run into is that quantum models are really hard to train. There's this thing called the **"Barren Plateau"** problem, which is basically a super-powered version of the "vanishing gradient" problem we learn about in Deep Learning. The optimizer just gets lost and stops learning.

Instead of just using Gradient Descent, I want to try using **Genetic Algorithms (GAs)** to "evolve" the quantum circuits.

How I want to build it:

1. **Start Random:** Create a bunch of random quantum circuit designs.
2. **Test Them:** See which ones actually learn the data best (this is the "fitness" part).
3. **Mix and Match:** Take the best designs, swap some of their "gates" (like DNA crossover), and create a new generation.
4. **Repeat:** Keep doing this until we find a circuit structure that doesn't get stuck and actually solves the problem.

4. Why this matters for the real world

If we can use these "Evolutionary Searchers" to find the best quantum circuits, we could solve some massive problems:

- **Navigation:** Optimizing traffic or flight paths in ways classical computers struggle with.
- **Medicine:** Helping scientists understand protein folding to create new drugs.
- **Math:** Searching for proofs or solutions in huge "logical trees."

5. The Reality Check (Being Honest)

I'm still an undergrad, and I know this stuff is hard. Simulating even 5 or 6 qubits on my laptop is slow, and real quantum computers are still really glitchy. But I don't think that's a reason to wait.

We aren't trying to beat Supercomputers today; we're trying to find the **optimizers of the future**. If we can prove that "evolving" a quantum circuit works better than just "training" one, we're ready for when the hardware eventually catches up.

Comparison Table

Feature	Normal AI (MLP)	My Quantum Idea
How it learns	Gradient Descent	Evolution / Genetic Algo
Math Space	Normal 2D/3D vectors	High-dimensional Quantum Space
Main Problem	Gets stuck in local minima	Simulation is slow
Best For	Recognizing images/text	Searching huge possibilities